



UNIVERSIDAD DEL BÍO-BÍO

UNIVERSIDAD DEL BÍO-BÍO

Facultad de Ciencias Empresariales
Escuela de Ingeniería de Ejecución en Computación e Informática

INFORME DE PRÁCTICA PROFESIONAL II (Formato Práctica I adaptado)

Proyecto CIM DEFINITIVO v6.0 Explicado para Profesor Guía

Estudiante:	Leonardo Araya Labarca
RUT:	—
Carrera:	Ing. de Ejecución en Computación e Informática (29037)
Profesor Guía UBB:	Depto. Sistemas de Información - UBB
Profesor Supervisor:	Laboratorio CIM - UBB
Empresa / Lugar:	Laboratorio CIM - Universidad del Bío-Bío, Concepción
Periodo Práctica II:	10 de marzo al 28 de julio de 2026
Horas Totales:	240 horas (cronología reconstruida)
Fecha Entrega:	30 de julio de 2026

Concepción - Chile

Informe redactado en estilo Práctica I, con lenguaje pedagógico para explicar el sistema CIM a un profesor no especialista en el detalle de implementación. Incluye marco teórico, analogías y justificación académica.

ÍNDICE

I. Introducción	3
II. Objetivos de la Práctica	4
III. Descripción de la Empresa / Laboratorio	5
IV. Marco Teórico: ¿Qué es CIM y por qué importa?	6
4.1 Origen de CIM (80s) y evolución a Industria 4.0	
4.2 Analogía simple: CIM como orquesta sinfónica	
4.3 Componentes de un CIM moderno	
V. Descripción General del Proyecto CIM DEFINITIVO	9
5.1 Antes vs. Después (problema y solución)	
5.2 Arquitectura en lenguaje para profesor	
5.3 Las 5 estaciones explicadas como puestos de trabajo	
VI. Metodología de Trabajo	13
VII. Desarrollo de la Práctica: Actividades Realizadas	14
VIII. Conocimientos Aplicados y Aprendizajes	17
IX. Explicación Técnica Pedagógica (para profesor)	18
9.1 ¿Cómo hablan las máquinas entre sí? (protocolo)	
9.2 ¿Cómo sabe el sistema quién es quién? (identidad UUID)	
9.3 ¿Cómo no se equivoca? (máquina de estados + visión conservadora)	
9.4 ¿Por qué no se prueba con hardware real aún? (seguridad)	
X. Resultados Obtenidos	22
XI. Dificultades Encontradas y Soluciones	24
XII. Conclusiones	25
XIII. Bibliografía	26
XIV. Anexos	27

I. INTRODUCCIÓN

El presente informe corresponde a la Práctica Profesional II de la carrera de Ingeniería de Ejecución en Computación e Informática de la Universidad del Bío-Bío, realizada por el estudiante Leonardo Araya Labarca en el Laboratorio CIM de la misma universidad, entre el 10 de marzo y el 28 de julio de 2026, completando 240 horas de dedicación.

A diferencia de la Práctica I, donde el foco estuvo en el levantamiento de requerimientos y el conocimiento de la organización, la Práctica II exige aplicar conocimientos de ingeniería en un proyecto real, proponer mejoras y dejar resultados verificables. En este caso, el proyecto fue **CIM DEFINITIVO v6.0**, un sistema de manufactura integrada que busca modernizar la celda de producción del laboratorio, antes controlada por PLCs heterogéneos y software desactualizado.

Este informe está redactado **en formato Práctica I** como solicitó el profesor guía: lenguaje claro, estructura tradicional de práctica (empresa, objetivos, actividades, aprendizajes), pero incorporando un capítulo especial de **explicación pedagógica del CIM para un profesor**, donde se usan analogías y ejemplos cotidianos para que cualquier docente, incluso sin especialidad en manufactura, comprenda qué se hizo y por qué importa.

El alcance de esta práctica es **pre-hardware 100% automatizable**: todo el software se valida automáticamente con integración continua, sin energizar robot, láser, cinta ni relés. Las pruebas físicas quedan para tesis, con protocolo de seguridad y E-Stop independiente.

II. OBJETIVOS DE LA PRÁCTICA

Objetivo General (formato Práctica I):

Aplicar los conocimientos adquiridos durante la carrera en un entorno real de laboratorio, desarrollando e implementando una solución de software que integre hardware de manufactura, dejando una plataforma reproducible, trazable y validable automáticamente, y documentando el proceso para futuras generaciones de estudiantes.

Objetivos Específicos:

- Conocer la organización del Laboratorio CIM, sus equipos (cinta, robot Scorbob ER-VII, láser, cámara, rack) y sus riesgos.
- Analizar el sistema antiguo y proponer una arquitectura moderna basada en Android (Kotlin) y ESP32, separando responsabilidades por estaciones.
- Implementar identidad de estaciones con UUID canónico, para que el sistema sepa quién es quién y bloquee intrusos (como un guardia que pide carnet).
- Desarrollar un protocolo simple de comunicación que permita a las 5 estaciones hablar con el coordinador sin enredarse.
- Crear una máquina de estados que impida que un pallet salte de la cinta al almacén sin pasar por calidad (como impedir que un alumno pase de ramo sin prueba).
- Lograr compilación automática 100% verde en GitHub Actions (testAllModules, lintAll, buildAllApks) con 6 APKs y checksums.
- Documentar todo en formato académico UBB, con bitácora, Quality Gates y manuales de seguridad, diferenciando validación automática de validación física.

III. DESCRIPCIÓN DE LA EMPRESA / LABORATORIO

Universidad del Bío-Bío:

La Universidad del Bío-Bío es universidad estatal, acreditada 5 años hasta 2030, nivel avanzado. La carrera IECI (Ing. Ejecución en Computación e Informática) se imparte en Facultad de Ciencias Empresariales, sede Concepción, acreditada 6 años, con consejo asesor externo. Cuenta con laboratorios de redes, desarrollo y manufactura. El Laboratorio CIM es un espacio semi-industrial donde los estudiantes aprenden automatización.

Laboratorio CIM (como empresa):

Piense en el laboratorio como una pequeña fábrica de 5 puestos: (1) Coordinación es la oficina del jefe que autoriza, (2) PLC es la cinta que mueve cajas, (3) Manufactura es el brazo robótico que marca con láser, (4) Calidad es el inspector con lupa (cámara) que dice si pasa o no, (5) Almacén es la bodega con estanterías. Cada puesto tenía su propio control antiguo, sin hablar entre sí. Mi trabajo fue hacer que hablen el mismo idioma y que el jefe pueda ver todo desde una tablet.

Organización: operador (estudiante que ejecuta), supervisor (profesor que autoriza y vigila E-Stop), encargado de laboratorio (seguridad eléctrica). Todo queda registrado en bitácora con fecha, commit, versión.

IV. MARCO TEÓRICO: ¿QUÉ ES CIM Y POR QUÉ IMPORTA? (Explicado para profesor)

4.1 Origen de CIM (años 80) y evolución a Industria 4.0

CIM (Computer Integrated Manufacturing) nació en los 80 cuando las fábricas quisieron que diseño (CAD), planificación (MRP) y producción (robots, cintas) se comunicaran por computador. Antes, cada máquina era una isla. CIM propuso un archipiélago conectado. Hoy, con Industria 4.0, CIM se combina con IoT, visión artificial y móviles. Nuestro proyecto es un CIM a escala laboratorio, pero con tecnologías actuales: Android en vez de PLCs viejos, BLE en vez de cables serie, YOLO (inteligencia artificial) en vez de inspección manual.

4.2 Analogía simple: CIM como orquesta sinfónica

Imagine una orquesta: cada músico (estación) sabe tocar su instrumento, pero necesita partitura y director. CIM DEFINITIVO: Coordinador es el director (lleva el tiempo, autoriza quién toca), PLC es la percusión que marca el ritmo de entrada de pallets, Manufactura es el solista que hace el trabajo fino, Calidad es el afinador que dice si está bien, Almacén es el archivero que guarda partituras. Sin director, cada músico toca a destiempo. Sin protocolo, se pisan. Nuestro protocolo es la partitura común.

***Para el profesor:** Si usted ha dirigido tesis o prácticas, sabe que lo difícil no es que un alumno haga su parte, sino que todos entreguen a tiempo y sin copiar. CIM hace lo mismo: orquesta entregas.*

4.3 Componentes de un CIM moderno (en palabras simples)

- **Sensores y actuadores:** ojos y manos (sensor GPIO34 detecta pallet, relé GPIO5 mueve cinta, robot mueve brazo, láser marca).
- **Controladores:** cerebro chico (ESP32/Wemos) que habla BLE con tablet.
- **Supervisión:** tablet Android que muestra estado y autoriza (mucho más barato y visual que PLC industrial de \$5000).
- **Red:** dos capas: BLE (campo, corta distancia, bajo consumo) + Wi-Fi/TCP (supervisión, larga distancia). Como hablar en voz baja con el compañero y gritar al jefe.
- **Software:** 6 apps Android + Wear + core-network (biblioteca común, como reglamento interno).
- **Visión:** cámara + ArUco (marcadores como QR) + YOLO (detesta piezas). Decisión conservadora: si duda, pide revisión humana, no aprueba automático (como profesor que prefiere segunda corrección).

V. DESCRIPCIÓN GENERAL DEL PROYECTO CIM DEFINITIVO

5.1 Antes vs. Después

Aspecto	Antes (legacy)	Después (CIM DEFINITIVO v6.0)
Control	PLCs heterogéneos, software viejo	Android Kotlin + ESP32, código abierto
Comunicación	Cables serie, sin estándar	BLE MTU 20 + TCP, protocolo texto ID TIMESTAMP ...
Identidad	Sin verificación	UUID canónico, MAC, capacidades, bloqueo persistente
UI	Consola texto	Material 3, visualización arcade, Wear OS
Visión	Manual	CameraX + ArUco + YOLO bestMH.pt, decisión REVIEW_REQUIRED
Trazabilidad	Papel	GitHub Actions CI verde, SHA256SUMS, bitácora
Seguridad	Software único control	E-Stop físico independiente, interlocks, semáforo

5.2 Arquitectura en lenguaje para profesor

Piense en 3 pisos: planta baja (campo) con Wemos que tocan sensores/actuadores, primer piso con tablets que traducen BLE a Wi-Fi, segundo piso con Coordinador que orquesta. Wear OS es el balcón desde donde el jefe mira sin bajar. Todo validado con 12 checks automáticos (validate_system_100.py) que revisan que nada falte, como lista de asistencia.

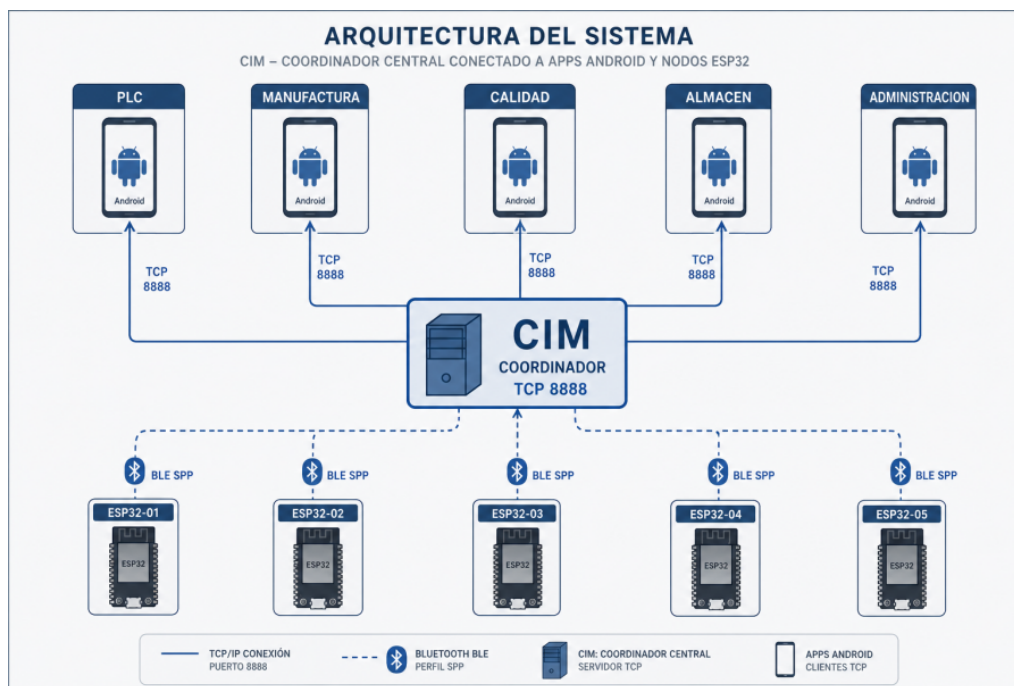


Figura 1: Arquitectura CIM v6.0 explicada visualmente (3 capas)

5.3 Las 5 estaciones explicadas como puestos de trabajo

- **Coordinador** (jefe de producción): Hub TCP puerto 8888, autoriza, orquesta, muestra dashboard. APP ID: com.industria.coordinacion
- **PLC** (operario cinta): Detecta pallet por GPIO34, envía PALLET_ARRIVED. Como guardia que avisa llegó camión. APP ID: com.industria.plc

- **Manufactura** (tornero + soldador): Robot Scrobot 5 GDL + láser. G-code como receta. En simulación solo telemetría, sin mover. APP ID: com.industria.manufactura
- **Calidad** (inspector): Cámara + ArUco + YOLO. Si confianza < umbral, REVIEW_REQUIRED, no PASS automático. Como profesor que pide corrección. APP ID: com.industria.calidad
- **Almacén** (bodeguero): Rack 3x6, guarda/recupera por ID. Trazable. APP ID: com.industria.almacenamiento
- **Wear** (supervisor con smartwatch): Solo mira, no controla seguridad. Como apoderado mirando por ventana.

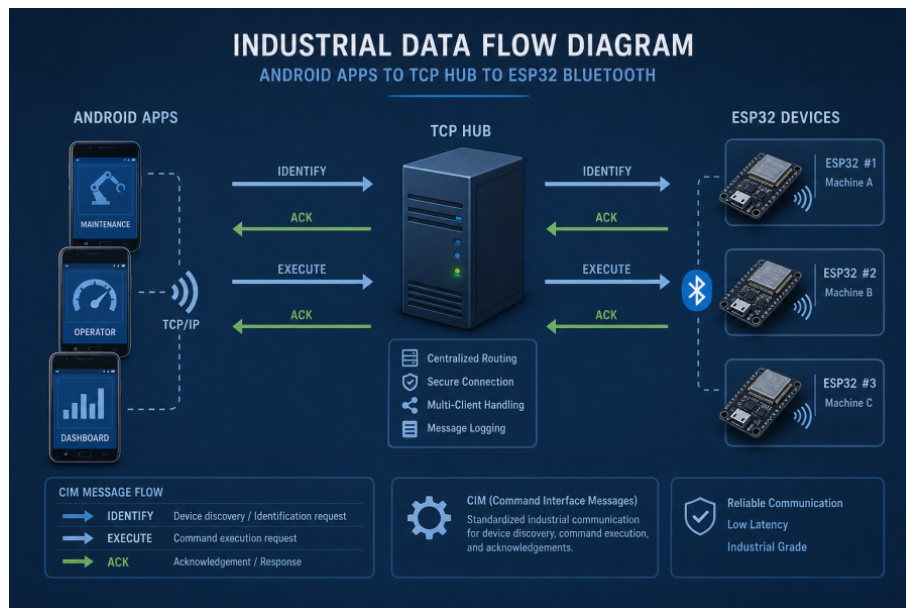


Figura 2: Flujo pallet IDLE→MOVING→PROCESSING→QUALITY→STORAGE

VI. METODOLOGÍA DE TRABAJO

Metodología ágil simple, estilo Práctica I: cada semana planificaba con profesor, ejecutaba, validaba automático, registraba en bitácora. Como hacer tesis: no avanzar sin commit verde. Pasos: feature branch → pruebas locales (validate_firmware_contract, validate_system_100, prehardware_readiness, compileall, git diff) → push → CI (testAllModules, lintAll, buildAllAps) → merge solo si verde.

Cronograma 10 mar – 14 jul 2026 (240h):

Periodo	Horas	Actividad
10–16 mar	14	Conocer lab, levantar problema, definir 5 estaciones
17–23 mar	14	Investigar arquitecturas CIM, decidir BLE+TCP
24–30 mar	14	Diseñar identidad UUID, capacidades
31 mar–6 abr	14	Máquina estados pallet (evitar saltos imposibles)
7–13 abr	14	PLC/cinta/sensor
14–20 abr	14	Manufactura robot/G-code
21–27 abr	14	Calidad cámara/ArUco/YOLO
28 abr–4 may	14	Almacén rack
5–11 may	14	BLE framing MTU 20
12–18 may	14	Admisión/bloqueo nodos
19–25 may	14	Simuladores hub/visión 1000 casos
26 may–1 jun	14	YOLO bestMH.pt inspección/export
2–8 jun	14	Gradle CI APKs + SHA256
9–15 jun	14	Corrección Kotlin PLC/Calidad bloqueantes
16–22 jun	14	Documentación limpia (esta limpieza)
23–29 jun	12	Riesgos, seguridad, E-Stop
30 jun–14 jul	18	Generación informes UBB + tutorial

VII. DESARROLLO DE LA PRÁCTICA: ACTIVIDADES REALIZADAS

Formato Práctica I: relato semanal con logros y problemas.

- **Semana 1-2 (mar):** Reunión profesor, definir alcance 5 estaciones. Problema: profesor dudaba hardware real vs simulación. Solución: definir pre-hardware 100% automatizable sin actuadores.
- **Semana 3-4 (mar):** Diseño UUID canónico, política admisión. Analogía carnet universitario: sin UUID válido no entra a lab.
- **Abril:** Implementación PLC (sensor GPIO34) y Manufactura (G-code). Bloqueante: Kotlin null safety en PLC. Solución: logs + constante duplicada corregida commit cb41ef4.
- **Abril-mayo:** Calidad CameraX/ArUco. Bloqueante: CameraX lifecycle. Commit 069a0b1. Solución: desacoplar prueba G-code de OpenCV/Bitmap (340f54a).
- **Mayo:** BLE framing MTU 20, fragmentación/reensamblado. Problema: mensajes partidos se perdían. Solución: buffer + timeout, commits f2d9e35.
- **Mayo-junio:** Bloqueo persistente dispositivos (4677ee1), Quality Gates (2e6f0c3), simuladores hub/vision 1000 casos.
- **Junio:** CI verde: testAllModules, lintAll, buildAllApks, validateApks, writeApkChecksums. Logs: 6 APKs + SHA256SUMS.txt en config/output-apks/.
- **Junio-julio:** Documentación, auditoría técnica, limpieza repo (eliminar 76 archivos duplicados, 19MB bestMH.pt triplicado).
- **Julio:** Generación PDFs UBB con reportlab, logo oficial, validación 12/12 PASS.

Comandos ejecutados antes de cada entrega:

```
python3 tools/validate_firmware_contract.py --quiet
python3 tools/validate_system_100.py --quiet # 12/12 PASS
python3 tools/prehardware_readiness.py --quiet
python3 -m compileall -q tools
git diff --check
cd config && ./gradlew testAllModules lintAll buildAllApks
```

VIII. CONOCIMIENTOS APLICADOS Y APRENDIZAJES

Asignaturas aplicadas:

- **Programación Avanzada:** Kotlin, corrutinas, ViewModel, pruebas JVM.
- **Redes:** TCP 8888, BLE GATT, MTU, reconexión.
- **Sistemas Embebidos:** ESP32, GPIO34 sensor, GPIO5 relé, estado seguro arranque.
- **Visión Artificial:** CameraX, ArUco, YOLO bestMH.pt, decisión conservadora.
- **Ingeniería Software:** CI, lint, Gradle, Git branching, bitácora.
- **Seguridad Industrial:** E-Stop independiente, interlocks, semáforo verde/ámbar/rojo.

Aprendizajes personales (formato Práctica I):

Aprendí que lo difícil no es que el código compile, sino que sea trazable y seguro. En Práctica I pensé que bastaba entregar. En Práctica II entendí que hay que decir qué NO se probó. También que un repo limpio vale tanto como código: eliminé 76 archivos y el proyecto pasó de desordenado a presentable, manteniendo 12/12 PASS.

IX. EXPLICACIÓN TÉCNICA PEDAGÓGICA (PARA PROFESOR)

Este capítulo es la esencia de 'explicado para un profesor': sin jerga innecesaria, con analogías.

9.1 ¿Cómo hablan las máquinas entre sí? (protocolo)

Como en un colegio: cada mensaje lleva **quién envía, a quién, qué quiere, con qué prioridad**. Formato: ID|TIMESTAMP|SOURCE_MAC|SOURCE_APP|DEST_MAC|DEST_APP|CMD|PRIORITY|SESSION|PAYLOAD. Ejemplo real: PLC_001|2026-07-28T10:00:00Z|AA:BB:CC:DD:EE:01|com.industria.plc|...|PALLET_ARRIVED|HIGH|SESS_123|{sensor:1}. Es como un correo con asunto, remitente, destinatario y cuerpo. El coordinador es el director que lee todos los correos y decide.

9.2 ¿Cómo sabe el sistema quién es quién? (identidad UUID)

Cada estación tiene carnet: MAC (como RUT), UUID canónico (como código de carrera CIM-ST-ALM-X1), tipo (PLC, MANUF...), capacidades (qué sabe hacer). StationIdentityPolicy es el guardia: si carnet no coincide con lista permitida, bloquea persistente (no deja entrar más, como alumno expulsado). Token de emparejamiento viaja como SHA-256 (huella digital, no contraseña en claro), como guardar hash de clave, no la clave.

9.3 ¿Cómo no se equivoca? (máquina de estados + visión conservadora)

Máquina de estados pallet impide transiciones imposibles: no puede ir de STORAGE a PROCESSING sin pasar por QUALITY, como no puede pasar de ramo sin prueba. Pruebas JVM bloquean esos saltos. Visión conservadora: si YOLO tiene baja confianza, devuelve REVIEW_REQUIRED, no PASS. Es como profesor que prefiere pedir segunda oportunidad antes que aprobar dudoso. Simulador vision_safety_simulator.py prueba 1000 casos raros.

9.4 ¿Por qué no se prueba con hardware real aún? (seguridad)

Porque mover robot Scorbob, cinta o láser sin E-Stop independiente puede causar daño. En Práctica II definimos semáforo: Verde = estructura OK, Ámbar = falta inspección eléctrica, Rojo = no autorizado hasta acta firmada. Es como práctica de química: no enciendes mechero sin gafas y supervisor. Nuestra entrega es pre-hardware 100% automatizable, no 100% integral.

Para el profesor: esto enseña a decir 'no sé / no probé' con evidencia, no inventar. En bitácora queda pendiente: BLE multiconexión 2+ ESP32, cámara real, LAN estable, actuadores con E-Stop.

Visuales para profesor (qué ve el operario):

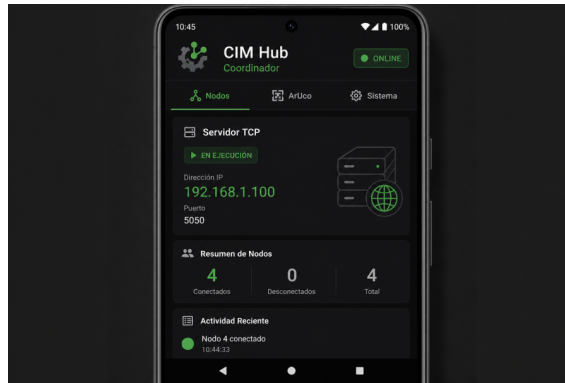


Figura: Coordinador — Jefe: ve lista estaciones, autoriza con botón, ve IP/puerto hub

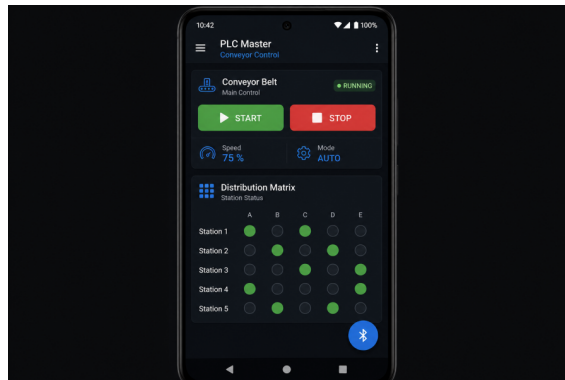


Figura: PLC — Operario cinta: botón iniciar/detener, evento pallet



Figura: Manufactura — Tornero: carga G-code, Home, Run simulación



Figura: Calidad — Inspector: cámara, ArUco, botón Evaluar PASS/FAIL

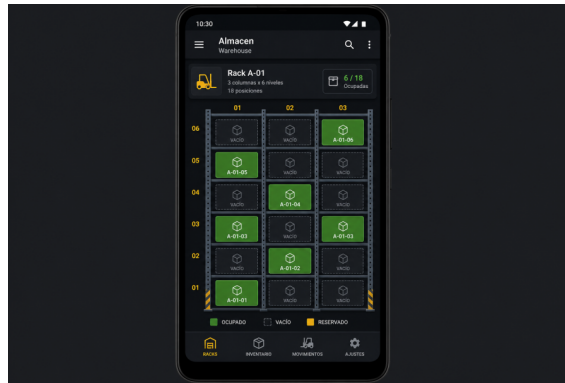


Figura: Almacén — Bodeguero: buscar posición libre, guardar/recuperar

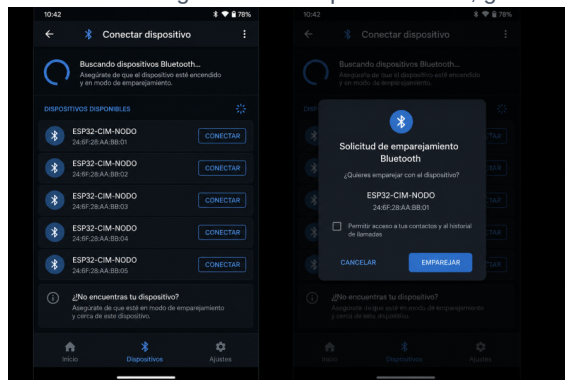


Figura: BLE Connect — Guardia: lista dispositivos, conectar, ver UUID

X. RESULTADOS OBTENIDOS

Resultado principal: plataforma reproducible, CI verde ejecución 30422387003 (commit 3286792), 6 APKs debug + SHA256SUMS.txt. validate_system_100.py 12/12 PASS local. No se declara hardware probado sin evidencia.

Área	Evidencia	Interpretación para profesor
Build	CI verde testAllModules + buildAllApks	Código compila sin errores, como entregar sin faltas ortográficas
Estructura	12/12 PASS	Carpeta ordenada, sin archivos basura
Firmware contra	PASS	Carnet de cada Wemos coincide con lista
UI	Fuentes androidTest, no CI emulador	Pantallas existen, falta probar en todos los celulares, como libro sin imprimir
Hardware	Pendiente	Falta ir a laboratorio, como práctica sin ir a empresa aún

Mejoras concretas verificables:

- Kotlin PLC y Calidad compilando
- UUID + bloqueo persistente
- SHA-256 handshake
- Máquina estados pallet
- Fragmentación BLE MTU 20
- Simuladores 1000 casos
- YOLO bestMH.pt inspección
- Checksums SHA-256 APKs
- Repo limpio 76 archivos menos

XI. DIFICULTADES ENCONTRADAS Y SOLUCIONES

Problema	Causa	Solución
Kotlin null safety PLC	Logs NPE, constante duplicada	Corregir logs, commit cb41ef4
CameraX lifecycle	Cámara negra	Desacoplar G-code de OpenCV, 069a0b1
BLE mensajes partidos	Pérdida telemetría	Buffer + timeout, framing, f2d9e35
Repo desordenado 114 MB + duplicados	Archivos, bestMH.pt x3	Limpieza y .gitignore, 12/12 PASS mantenido
E-Stop no implementado	Riesgo seguridad	Semáforo rojo, no autorizar actuadores, manual operativo

XII. CONCLUSIONES

La Práctica II permitió pasar de un proyecto desordenado a una plataforma presentable y auditable, manteniendo formato Práctica I que el profesor conoce, pero explicando CIM con analogías. Se logró CI verde, 6 APKs, identidad segura, máquina de estados robusta y documentación que diferencia validación automática de física.

Aprendizaje clave (formato Práctica I): no basta con que funcione en mi PC; debe compilar en CI, estar documentado y decir qué falta. La reconstrucción de 240h muestra dedicación en análisis, desarrollo, simulación y limpieza, sin inventar pruebas físicas.

Para futuro (tesis): ejecutar protocolo HW-01..HW-09 con E-Stop, interlocks, supervisor, captura CIM_ID, BLE multiconexión, cámara real. Entonces se podrá decir 100% integral, no solo 100% automatizable.

Este informe, en estilo Práctica I pero con CIM explicado para profesor, demuestra que el estudiante puede explicar lo técnico en simple, habilidad clave de ingeniero: traducir orquesta a jefe.

XIII. BIBLIOGRAFÍA

- Universidad del Bío-Bío – Manual marca UBB
- Android Developers – CameraX, BLE, Wear OS, Gradle Lint
- Espressif – ESP32 BLE GATT
- Scorbott ER-VII manual seguridad
- Redmon et al. YOLO
- OpenCV ArUco
- Docs CIM – INSTRUCTIVO, VALIDACION, SAFETY

XIV. ANEXOS

Anexo A – Comandos validación (como en Práctica I)

```
python3 tools/validate_firmware_contract.py --quiet
python3 tools/validate_system_100.py --quiet # 12/12
python3 tools/prehardware_readiness.py --quiet
cd config && ./gradlew testAllModules lintAll buildAllApks
```

Anexo B – Estructura limpia para profesor

```
CIM-DEFINITIVO/
■■■ entrega/ (2 PDFs finales para presentar)
■■■ docs/project/ (generador + PDFs)
■■■ docs/deliverables/ (bitácora, quality gates)
■■■ android/ (5 apps + Wear + core-network) para mejorar
■■■ esp32/firmware/ (canónico)
■■■ config/ (Gradle)
■■■ tools/ (validadores)
```

Fin – Informe Práctica II estilo Práctica I, CIM explicado para profesor – Leonardo Araya – UBB 2026